LAB EXERCISE: 4

Aim / Objective

Apply Latent Dirichlet Allocation (LDA) to a document collection and interpret the discovered topics using probabilistic and sampling-based methods. The implemented Python pipeline preprocesses text, builds a document-term count matrix, trains LDA through collapsed Gibbs sampling, and exports topic-word and document-topic probabilities with a visual results page.

Expected Outcomes Addressed

• **Preprocess text data:** lowercase, tokenize alphabetic words, remove stopwords and tokens shorter than three letters.

• **Build a document-term representation:** create a sorted vocabulary and a 32 x 195 integer count matrix.

• **Train an LDA model:** sample latent topic assignments for 400 sweeps with fixed random seed 42 and symmetric Dirichlet priors.

• **Interpret topic-word distributions:** inspect high-probability words with representative documents and describe four learned topics.

Tools & Technologies Used

Category	Tool / version	Purpose
Language / runtime	Python 3.11.9	Execution, CSV and JSON processing, regular expressions and tests
Numerical computation	NumPy 2.3.5	Count arrays, random sampling, probability distributions and model export
Frontend	HTML5 / CSS3	Static topic charts, representative documents and output links
Report generation	ReportLab 5.0.1	PDF layout, tables and vector charts
Verification	unittest (standard library)	Four automated correctness and validation checks

The training pipeline requires NumPy only. ReportLab is an optional dependency for regenerating this PDF. No hosted API, pretrained model or downloaded dataset is used.

Dataset / Corpus Description

The collection contains 32 original educational documents, eight each about space exploration, cricket, healthcare, and computing. Documents were authored for this demonstration and are stored in data/corpus.csv. They are deliberately separable; this is not a real-world benchmark or evidence of general-purpose topic-modelling quality.

Input columns are id, category and text. Only text is used for training. Category values are used after training to describe representative documents. For custom data, id and text are required; category is optional. Empty post-preprocessing documents and duplicate IDs are rejected.

Property	Observed value
Documents	32
Retained tokens / vocabulary	420 / 195
Count matrix	32 documents x 195 vocabulary terms
Topic count	4 (chosen for demonstration scope)
Provenance	Original local corpus; no external dataset

Sample raw record (computing_01): The software developer wrote code for a computer application. The program processed data and stored results on a network server.

Preprocessed tokens: software developer wrote code computer application program processed data stored results network server

System Architecture / Pipeline Design

Stage	Input and operation	Output
1. Input validation	Read UTF-8 CSV; validate IDs and text	Document records
2. Preprocessing	Lowercase, regex tokenization and stopword filtering	Clean token lists
3. Representation	Build vocabulary and integer token counts	Document-term matrix
4. Sampling	Initialize assignments; Gibbs resample token topics	Posterior count states
5. Estimation	Burn-in; average retained theta and phi samples	Normalized distributions
6. Interpretation	Rank words and representative documents	CSV tables, HTML and report

The pipeline uses bag-of-words counts. Word order and grammatical structure are not modelled. Stemming is omitted to preserve readable terms, so related morphological forms can remain separate vocabulary entries.

Backend Implementation

6.1 Approach / Algorithm

LDA assumes that each document has a distribution over latent topics and each topic has a distribution over vocabulary words. Symmetric Dirichlet priors use alpha = 0.1 for document-topic mixtures and beta = 0.01 for topic-word distributions.

For each token, remove its current assignment from all counts. Then sample a topic using the following conditional, where the counts exclude the token currently being updated:

```
P(z_i=k | rest) proportional to
(n_dk + alpha) * (n_kw + beta) / (n_k + V*beta)
```

Here n_dk counts topic k in document d, n_kw counts word w in topic k, n_k is the total count for topic k, and V is vocabulary size. After sampling, increment the new counts. Repeat for 400 complete corpus sweeps.

The first 200 sweeps are burn-in. Retain every tenth subsequent state, producing 20 retained samples. Estimate theta and phi for each retained state, then average the estimates. This reduces dependence on one final assignment but does not establish convergence.

```
theta[d,k] = (n_dk + alpha) / (N_d + K*alpha)
phi[k,w]   = (n_kw + beta) / (n_k + V*beta)
```

6.2 Key Code Snippets

```
# Text preprocessing
def preprocess(text):
    return [w for w in re.findall(r'[a-z]+', text.lower())
            if len(w) > 2 and w not in STOP]

# Resampling, after decrementing the current counts
p = (nd[d] + alpha) * (nw[:, w] + beta)
p = p / (nk + vocab_size * beta)
z = rng.choice(k, p=p / p.sum())
assignments[d][i] = z
nd[d, z] += 1
nw[z, w] += 1
nk[z] += 1
```

The complete implementation is in lda_lab.py. The exported compressed model contains theta, phi, vocabulary, document IDs, alpha and beta. Raw text and category labels are not embedded in the model file.

Frontend / UI Implementation

7.1 Framework Choice

The frontend is a standalone HTML5/CSS3 results page generated by lda_lab.py. Open outputs/report.html in a browser after training. It runs locally without JavaScript, a server or an API key.

7.2 Interface Features

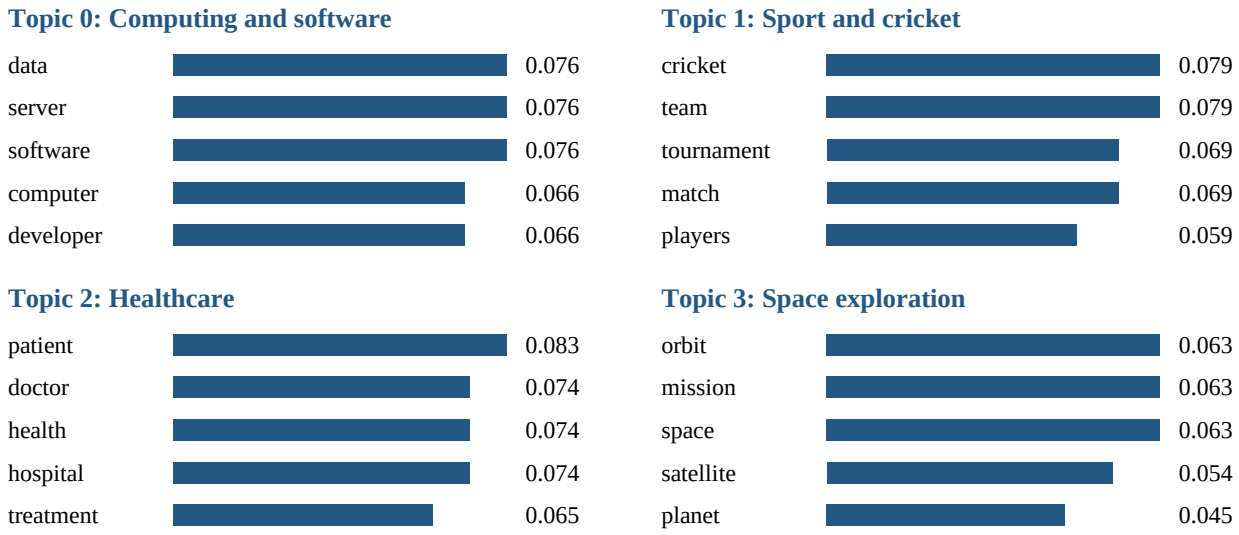
• Summary cards show corpus size, vocabulary size, number of topics and sampling sweeps.

• Four topic cards display the ten leading words and their probabilities. Bar lengths are scaled to the largest displayed probability within each topic.

• Three representative documents per topic show actual document-topic probabilities and original text.

• Links provide the interpretation report, topic-word table, document-topic probabilities, preprocessed corpus and run metrics.

Visual evidence: The chart below is generated directly from the saved model arrays. It is a vector results chart, not a browser screenshot. An automated screenshot of the local HTML page was blocked by the browser URL policy; a captured frontend screenshot remains to be added if required for submission.



Sample Input & Output

Item	Actual result
Input ID	computing_01
Input text	The software developer wrote code for a computer application. The program processed data and stored results on a network server.
Document-topic probabilities	Topic 0: 0.9701, Topic 1: 0.0075, Topic 2: 0.0112, Topic 3: 0.0112
Dominant topic	Topic 0: Computing and software

Results (Deliverable-Specific Outputs)

Topic	Interpretation	Top words and P(word | topic)
0	Computing and software	data (0.076), server (0.076), software (0.076), computer (0.066), developer (0.066), network (0.057)
1	Sport and cricket	cricket (0.079), team (0.079), tournament (0.069), match (0.069), players (0.059), runs (0.059)
2	Healthcare	patient (0.083), doctor (0.074), health (0.074), hospital (0.074), treatment (0.065), medical (0.064)
3	Space exploration	orbit (0.063), mission (0.063), space (0.063), satellite (0.054), planet (0.045), spacecraft (0.045)

Interpretations are assigned after training by examining words and representative documents. Topic IDs are arbitrary and can change between runs. Top-word probabilities do not sum to one because only part of the full vocabulary is shown.

Deliverable	Generated file
Preprocessed corpus	outputs/preprocessed_corpus.csv
Document-term counts	outputs/document_term_matrix.csv
Saved model distributions	outputs/lda_model.npz
Top ten words per topic	outputs/topic_words.csv
Document-topic mixtures	outputs/document_topics.csv
Interpretation report	outputs/interpretation_report.md
Visual results page	outputs/report.html
Run metrics	outputs/metrics.json

Evaluation / Analysis

Check / metric	Observed result	Meaning
Automated tests	4 passed	Preprocessing, deterministic sampling, normalized positive distributions, invalid parameters and empty-document rejection
Count invariants	Passed	Counts remain nonnegative; all 420 tokens are conserved
Probability normalization	Passed	Each theta and phi row sums to one
Training reconstruction perplexity	42.0891	In-sample word reconstruction; not held-out evaluation

Perplexity reconstructs observed words using averaged theta and phi. It is not held-out performance, accuracy or coherence. Precision, recall and F1 are not claimed: no supervised prediction benchmark was defined.

Challenges Faced & Solutions

• **Dataset availability:** No corpus was supplied with the rubric. A small original corpus was included with transparent provenance and a CSV input option for replacement data.

• **Sampling correctness:** Updating a token without first decrementing its counts biases its conditional. The implementation removes the current assignment before drawing and restores counts afterward; conservation checks validate the result.

• **Reproducibility:** Random assignments can produce different topics. A fixed seed, pinned NumPy dependency, recorded settings and deterministic-sampling test make the submitted run reproducible in the tested environment.

• **Interpretation:** Topic IDs carry no semantic labels. Top words and representative document text are reviewed jointly; category metadata is used only after fitting.

• **Misleading evaluation:** Training fit can look optimistic. The reconstruction metric is explicitly marked in-sample, with no unsupported accuracy claims.

Limitations and possible extensions

The corpus is small and intentionally clean, so separation is easier than for real news or research articles. Bag-of-words ignores word order, negation and semantic context. Fixed stopwords may remove useful domain terms; morphological variants split counts. Four topics were chosen for the demonstration rather than selected through model comparison. One finite Gibbs chain does not demonstrate convergence or sampling uncertainty.

Extensions include a larger real-world corpus, multiple independent chains, topic-count and prior sensitivity studies, held-out inference and evaluation, and topic coherence analysis. For arbitrary custom CSVs, representative-document category descriptions must be reviewed rather than treated as learned ground truth.

Conclusion

A complete LDA topic-modelling pipeline was implemented and executed using collapsed Gibbs sampling. The 32-document demonstration produced four interpretable themes: computing and software, sport and cricket, healthcare, and space exploration. The submission includes cleaned text, count representation, model arrays, probability tables, a visual frontend and reproducibility checks.

The exercise demonstrates how document-topic mixtures and topic-word probabilities are inferred from token counts under Dirichlet priors. It also distinguishes descriptive topic interpretation from predictive evaluation, and documents the limitations of a small corpus and a single sampling run.

Project Repository (GitHub Link)

https://github.com/moulyad04/Discovering-topics-with-LDA

Field	Value
Repository	moulyad04/Discovering-topics-with-LDA
Submission branch	main
Visibility	Check repository settings on GitHub
Submission contents	Code, corpus, generated results, tests, README, dependencies and this PDF

Setup and backend execution

```
python -m venv .venv
.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python lda_lab.py
python -m unittest discover -s tests -v
```

Frontend execution: open outputs/report.html in a local browser. For a custom corpus, run python lda_lab.py --input my_documents.csv --output outputs_custom --topics 4 --iterations 600 --seed 42.

PDF regeneration: install requirements-report.txt, then run python tools/build_report.py from the repository root. Regenerating uses the current saved model and metrics.

References

• Blei, D. M., Ng, A. Y., and Jordan, M. I. (2003). Latent Dirichlet Allocation. Journal of Machine Learning Research, 3, 993-1022. jmlr.org/papers/v3/blei03a.html

• Griffiths, T. L., and Steyvers, M. (2004). Finding scientific topics. PNAS, 101 (Suppl. 1), 5228-5235. DOI: 10.1073/pnas.0307752101. Sampling approach used as methodological background.

• NumPy documentation: numpy.org/doc/stable/. Numerical arrays and random sampling.

• Python documentation: docs.python.org/3/. CSV, regular expressions, pathlib and unittest.

Code and corpus were prepared for this exercise. No external dataset or copied third-party code was used. The supplied lab rubric defines the task; the supplied Lab 1 report defines the report section order and visual format.

ADVANCE NLP (AML23702)
LAB EXERCISE EVALUATION SHEET

Student details	Value
Student Name	MOULYA D
USN	1GA23AI026
Lab Exercise No.	4
Experiment Title	Discovering Topics with Latent Dirichlet Allocation

Marks Awarded

Sl. No.	Evaluation Criteria	Max Marks	Marks Obtained
1	Problem Understanding & Scope	5	
2	Pipeline Design & Methodology	10	
3	Implementation Correctness	10	
4	Use of NLP Tools / Models	5	
5	Output Quality / Task Performance	10	
6	Analysis & Interpretation	5	
7	Documentation & Deliverables	2.5	
8	Presentation & Reproducibility	2.5	
	Total	50	

Overall Comments

Date of Submission	Signature of Faculty